COLEGIO SECUNDARIO SAN JUAN

Arquitectura y funcionamiento

C?mo se conectan React, la API REST en Go y SQLite

INTERFAZ React + Vite	API Go + Chi	DATOS SQLite

Gu?a pr?ctica de capas, flujo de inicio de sesi?n, solicitudes, notas, base de datos y l?mites de integraci?n.

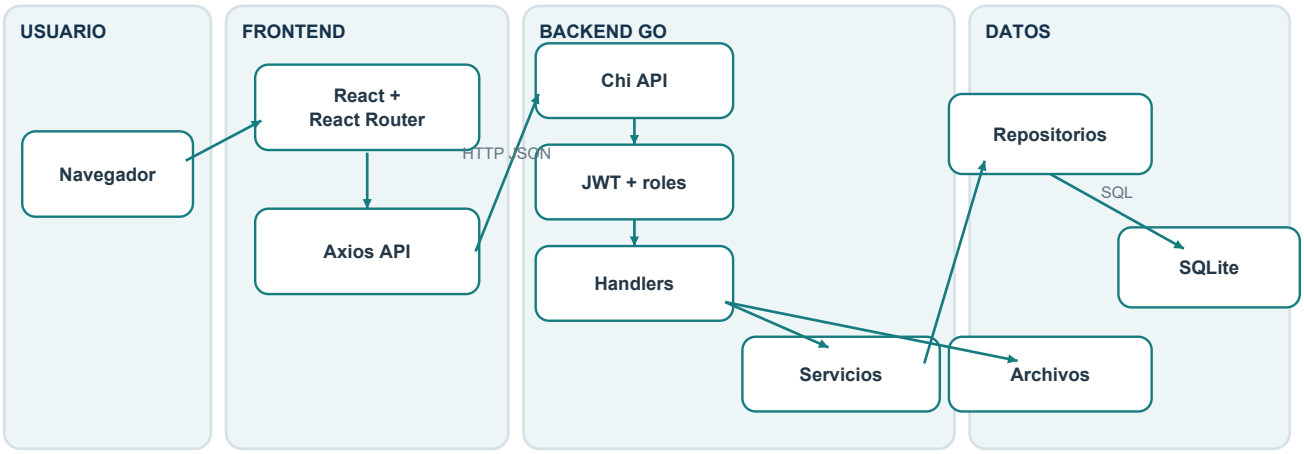
Basado en la arquitectura acordada para el proyecto y en la implementaci?n actual del repositorio.

Versi?n 1.0 | 9 de noviembre de 2026

Guía de cómo se conectan la interfaz React, la API REST en Go y la base de datos SQLite. Describe el comportamiento implementado en el repositorio y distingue las pantallas que todavía no persisten información.

Vista general

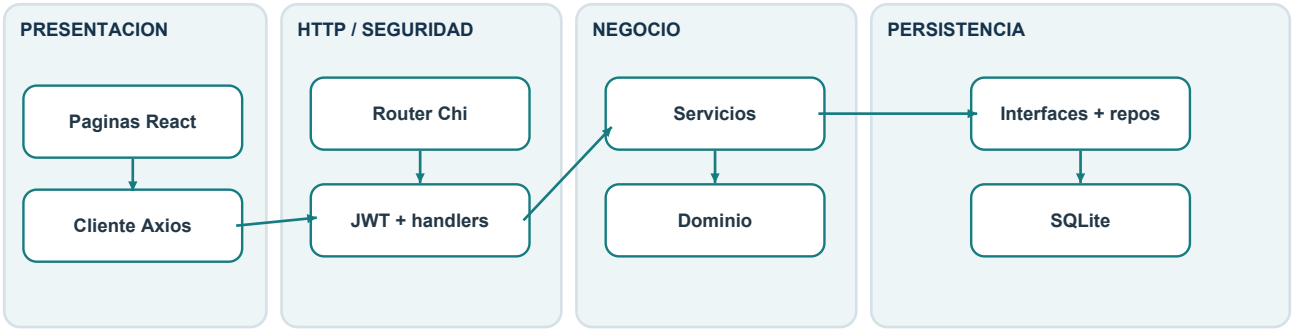
El sistema se ejecuta como dos procesos locales: Vite sirve la aplicación web en `http://localhost:5173` y el servidor Go atiende la API en `http://localhost:8080`. El navegador no abre la base de datos: toda lectura o escritura pasa por la API.



El navegador nunca abre SQLite directamente; todas las operaciones pasan por la API.

La aplicación también puede servir archivos cargados bajo `/uploads`. En desarrollo, Vite y Go son servidores independientes. `VITE_API_URL` permite cambiar el host base del cliente HTTP; si no se define, se usa `http://localhost:8080`.

Capas y responsabilidades



El arranque del backend conecta las implementaciones con sus contratos y configura el router.

Capa	Ubicación	Responsabilidad
Interfaz web	`frontend/src/features`, `components`, `layouts`	Presenta páginas públicas y módulos académicos; conserva el estado temporal de los formularios.
Rutas del frontend	`frontend/src/App.jsx`, `pages/AppRoutes.jsx`	React Router conecta las URL con sus páginas y limita rutas de interfaz según el rol cargado.
Cliente HTTP	`frontend/src/api/client.js`	Axios agrega `/api`, establece el tiempo de espera, adjunta el token guardado y convierte errores HTTP en mensajes.
Entrada HTTP	`backend/internal/handler`	Chi enruta solicitudes, decodifica datos y produce respuestas HTTP.
Seguridad transversal	`backend/internal/middleware`	Valida JWT y comprueba roles para las rutas protegidas. CORS, registro de solicitudes y recuperación de errores se configuran en el router.
Casos de uso	`backend/internal/service`	Coordina autenticación, solicitudes de matrícula, notas, historial y administración académica.

Capa	Ubicación	Responsabilidad
Dominio	`backend/internal/domain`	Define entidades y reglas de negocio, incluidos State y Memento.
Persistencia	`backend/internal/repository` y `repository/sqlite`	Expone contratos y los implementa mediante consultas y transacciones SQLite.
Esquema	`backend/migrations`	Aplica cambios SQL versionados y registra su ejecución en `schema_migrations`.

El punto de composición es `backend/cmd/api/main.go`: abre la base de datos, crea repositorios, servicios y handlers, los conecta con el router y arranca el servidor. La dirección de dependencias del caso de uso apunta a interfaces de repositorio; SQLite es el adaptador concreto.

Flujo de inicio de sesión

1. La página de acceso envía usuario y clave con `POST /api/auth/login`.
2. El handler delega la autenticación al servicio, que busca al usuario y compara la clave con su hash bcrypt.
3. Si la cuenta está activa y la clave coincide, el servidor firma un JWT con usuario, nombre, rol y vencimiento.
4. El frontend guarda el token y el perfil. El cliente Axios adjunta `Authorization: Bearer <token>` a solicitudes posteriores.
5. El middleware Go valida el token antes de entregar la solicitud al handler. `RequireRole` verifica si el rol puede ejecutar la ruta.

Las rutas públicas de solicitud de matrícula no exigen sesión. Los módulos de administración y notas sí requieren autenticación y, según la operación, un rol autorizado. La comprobación de ruta del frontend mejora la navegación, pero la autorización efectiva está en el backend.

Flujos funcionales

Solicitud pública de matrícula

La persona completa el asistente de tres pasos. El navegador envía los datos del estudiante, apoderado y, opcionalmente, un archivo mediante `POST /api/enrollments/public`. El backend valida y guarda la solicitud y puede exponer el documento desde `/uploads`. Secretaría o Dirección revisan solicitudes por las rutas protegidas `/api/enrollments`.

Una solicitud de admisión no es lo mismo que una matrícula académica anual. Son modelos distintos en la base de datos; la migración inicial puede convertir solicitudes ya aprobadas al modelo académico, pero aprobar una solicitud en tiempo real no crea automáticamente la matrícula académica.

Notas e historial

Docente o Dirección acceden a `/api/grades`. Al guardar una nota, el servicio valida la operación y el repositorio actualiza la calificación junto con su versión histórica en una transacción. Si el guardado del historial falla, también se revierte la modificación. Las rutas de historial, deshacer, rehacer y restauración están protegidas por rol.

El patrón Memento conserva versiones anteriores de las notas B1-B4 en `grade_mementos`. El esquema académico nuevo también contiene `notas` y `nota_historial`, pero esas tablas no equivalen a una API independiente para todo el modelo nuevo.

Dirección y catálogos

Las rutas `/api/director/*` permiten a Dirección consultar y administrar usuarios, asignaturas y asignaciones docentes. Las rutas `/api/academic/*` ofrecen datos de referencia para cuentas autenticadas, como secciones, cursos y asignaciones.

Datos y arranque de una instalación nueva

SQLite almacena usuarios, solicitudes, estudiantes, apoderados, pagos, notas e historiales. Al ejecutar el backend, `NewConnection` abre o crea el archivo indicado por `DB_PATH` —por defecto `app.db`—, aplica las migraciones incluidas en el binario y crea datos de demostración si la tabla de usuarios está vacía. Por eso no hace falta incluir una copia de la base de datos en Git para que una instalación limpia pueda iniciar sesión.

Las cuentas de demostración de la base nueva son `secretaria`, `director` y `profesor`; las claves están en la sección Credenciales de prueba y perfiles del README. Cambia las claves de prueba antes de cualquier uso real. El archivo SQLite es local y contiene datos que se registren durante el uso; no debe publicarse.

Qué está conectado y qué está pendiente

Módulo	Estado de integración
Inicio de sesión	Conectado a autenticación, JWT y control de roles del backend.
Solicitud pública y gestión de admisiones	Conectadas a las rutas de solicitudes y su persistencia SQLite.
Notas	Conectadas a las rutas B1-B4, historial y operaciones Memento.
Docentes y administración académica	Conectadas a las rutas de usuarios, asignaturas y asignaciones disponibles.
Asistencia	La pantalla puede cargar alumnos y preparar cambios locales; no existe una ruta API para guardar asistencias. El botón de guardado está deshabilitado.
Horarios	La pantalla consulta asignaciones; no hay una ruta que devuelva horas y aulas ni una API para guardar horarios. La vista no inventa esos bloques.
Modelo académico normalizado	Sus tablas y migraciones están preparadas; algunas operaciones aún no tienen rutas ni pantallas conectadas.

Estas limitaciones son deliberadas: una pantalla de previsualización no debe presentarse como si hubiera guardado datos.

Ejecución local

Con Go y Node.js instalados, abre dos terminales desde la raíz:

```
cd backend
go run ./cmd/api
```

```
cd frontend
npm ci
npm run dev
```

La interfaz queda en `http://localhost:5173`; la API en `http://localhost:8080`. `npm ci` vuelve a descargar las dependencias desde `package-lock.json`; no se sube `node_modules`.

Referencias del repositorio

- Diagrama de arquitectura y relaciones
- Contrato HTTP
- Estructura general y credenciales de prueba